

Sveučilište Jurja Dobrile u Puli
Fakultet informatike

Jakov Benedikt Ružić

Razvoj platformske računalne igre Run, Slavko, Run! u Unity okruženju

Završni rad

JMBAG: 0269158890, redoviti student
Studijski smjer: Preddiplomski sveučilišni studij Informatika

Predmet: Dizajn i programiranje računalnih igara
Znanstveno područje: Društvene znanosti
Znanstveno polje: Informacijske i komunikacijske znanosti
Znanstvena grana: Informacijski sustavi i informatologija

Mentor: prof. dr. sc. Tihomir Orehovački

Pula, rujan 2026.

IZJAVA O AKADEMSKOJ ČESTITOSTI

Ja, dolje potpisani Jakov Benedikt Ružić, kandidat za prvostupnika informatike, ovime izjavljujem da je ovaj Završni rad rezultat isključivo mogega vlastitog rada, da se temelji na mojim istraživanjima te da se oslanja na objavljenu literaturu kao što to pokazuju korištene bilješke i bibliografija. Izjavljujem da niti jedan dio Završnog rada nije napisan na nedozvoljeni način, odnosno da je prepisan iz kojega necitiranog rada, te da ikoji dio rada krši bilo čija autorska prava. Izjavljujem, također, da nijedan dio rada nije iskorišten za koji drugi rad pri bilo kojoj drugoj visokoškolskoj, znanstvenoj ili radnoj ustanovi.

Student
Jakov Benedikt Ružić

U Puli, rujan 2026.

IZJAVA O KORIŠTENJU AUTORSKOGA DJELA

Ja, Jakov Benedikt Ružić, dajem odobrenje Sveučilištu Jurja Dobrile u Puli, kao nositelju prava iskorištavanja, da moj završni rad pod nazivom „Razvoj platformske računalne igre Run, Slavko, Run! u Unity okruženju” koristi na način da gore navedeno autorsko djelo, kao cjeloviti tekst, trajno objavi u javnoj internetskoj bazi Sveučilišne knjižnice Sveučilišta Jurja Dobrile u Puli te preslika u javnu internetsku bazu završnih radova Nacionalne i sveučilišne knjižnice (stavljanje na raspolaganje javnosti), sve u skladu sa Zakonom o autorskom pravu i drugim srodnim pravima i dobrom akademskom praksom, a radi promicanja otvorenoga, slobodnoga pristupa znanstvenim informacijama.

Za korištenje autorskog djela na gore navedeni način ne potražujem naknadu.

Potpis

U Puli, rujan 2026.

Sažetak

U ovom radu opisan je razvoj dvodimenzionalne platformске igre „Run, Slavko, Run!” u razvojnom okruženju Unity. Igra pripada podžanru trkačkih platformera (engl. runner), u kojem se lik kreće naprijed automatski, a igrač upravlja isključivo skokom i posebnim mehanikama. Cilj igre je pobjeći iz zoološkog vrta kroz četiri razine, izbjegavajući prepreke i čuvara koji stalno slijedi igrača.

Težište rada je na projektiranju i podešavanju fizike kretanja. Detaljno je razrađena mehanika njihanja na užetu, izvedena preko zgloba DistanceJoint2D, te je matematički pokazano kako su širine jama odabrane tako da ih nije moguće preskočiti običnim skokom. Time mehanika njihanja dobiva stvarnu svrhu umjesto da bude ukras. Opisan je i postupak proceduralnog sastavljanja razina iz ponavljajućih segmenata, uz automatsku provjeru prohodnosti koja za svaku prepreku računa mjesto doskoka i preostali zalet.

Jednak pristup primijenjen je i na protivnika. Simulacijom potjere pokazano je da se razmak između čuvara i igrača ustaljuje na stalnoj vrijednosti, zbog čega čuvar u ispravnoj vožnji nikada ne može uhvatiti igrača, dok svaka pogreška igrača taj razmak trenutno smanjuje. Na temelju te analize sudar s preprekom pretvoren je u posrtanje koje ne oduzima život, nego oduzima vrijeme, čime protivnik dobiva priliku da kazni niz pogrešaka.

Rezultat je cjelovita igra s glavnim izbornikom, spremanjem napretka, sustavom života, pauzom, zvučnom podlogom i završnom animacijom, izgrađena u samostalnu izvršnu datoteku za Windows.

Ključne riječi: Unity, C#, 2D igra, platformer, fizika krutog tijela, proceduralno generiranje razina, dizajn igre

Abstract

This thesis describes the development of a two-dimensional platform game titled “Run, Slavko, Run!” using the Unity game engine. The game belongs to the runner subgenre, in which the character moves forward automatically and the player controls only jumping and special mechanics. The objective is to escape from a zoo across four levels while avoiding obstacles and a guard who continuously pursues the player.

The focus of the thesis is the design and tuning of movement physics. The rope-swing mechanic, implemented using a DistanceJoint2D, is described in detail, and it is shown mathematically that pit widths were chosen so that they cannot be cleared by an ordinary jump. This gives the swing mechanic a genuine purpose rather than making it decorative. The thesis also describes procedural level assembly from repeating segments, together with an automated traversability check that computes the landing position and remaining run-up for every obstacle.

The same approach is applied to the pursuing guard. A simulation of the chase shows that the gap between the guard and the player settles at a constant value, so the guard can never catch a player who makes no mistakes, while every mistake immediately closes that gap. Following that analysis,

colliding with an obstacle was redesigned to cost time rather than a life, so that the guard becomes the agent that punishes a sequence of mistakes.

The result is a complete game featuring a main menu, progress saving, a health system, a pause menu, a synthesized soundtrack and a victory animation, built into a standalone Windows executable.

Keywords: Unity, C#, 2D game, platformer, rigid body physics, procedural level generation, game design

Sadržaj

Sažetak.....	4
Abstract.....	4
Sadržaj.....	6
1. Uvod.....	8
2. Korištene tehnologije.....	10
2.1. Razvojno okruženje Unity.....	10
2.2. Programski jezik C#.....	10
2.3. Fizika krutog tijela u dvije dimenzije.....	10
2.4. Sustav za praćenje izvornog koda.....	11
2.5. Pomoćni alati za izradu sadržaja.....	11
3. Analiza žanra i postavljanje zahtjeva.....	12
3.1. Obilježja trkačkih platformera.....	12
3.2. Usporedive igre.....	12
3.3. Zahtjevi na igru.....	13
3.4. Koncept igre.....	14
4. Izrada igre.....	15
4.1. Struktura projekta.....	15
4.2. Arhitektura programskog rješenja.....	15
4.3. Kretanje igrača.....	17
4.4. Mehanika njihanja na užetu.....	18
4.4.1. Prikaz užeta.....	18
4.4.2. Pomoć igraču.....	18
4.5. Projektiranje jama.....	18
4.6. Protivnik.....	20
4.6.1. Prepreke kao izvor pritiska umjesto izravne štete.....	21
4.6.2. Odbacivanje nakon udarca.....	22
4.6.3. Nalet na posljednjoj razini.....	23
4.7. Sastavljanje razina.....	23
4.7.1. Provjera prohodnosti.....	23
4.7.2. Programsko stvaranje scena.....	24
4.8. Razlike među razinama.....	25
4.9. Predmeti koji se skupljaju i kontrolne točke.....	26
4.10. Grafika i pozadine.....	26
4.11. Zvuk.....	27
4.12. Korisničko sučelje.....	28
4.13. Izgradnja izvršne datoteke.....	29

4.14. Izgled gotove igre.....	30
5. Testiranje i podešavanje.....	34
5.1. Postupak provjere.....	34
5.2. Značajnije uočene pogreške.....	35
5.3. Dvije pogreške vrijedne posebnog opisa.....	36
5.3.1. Zadržavanje u jami.....	36
5.3.2. Prelomljeni tekst na pobjedničkom zaslonu.....	36
5.4. Podešavanje osjećaja upravljanja.....	36
6. Zaključak.....	38
Literatura.....	40
Popis slika.....	40
Popis tablica.....	41

1. Uvod

Razvoj računalnih igara jedno je od područja u kojem se programiranje, matematika i dizajn susreću izravnije nego drugdje. Odluka koja na papiru izgleda kao estetsko pitanje, primjerice koliko je široka jama u razini, u praksi je matematički problem: ako je jama uža od dometa skoka, mehanika predviđena za njezino prelaženje postaje nepotrebna, a ako je preširoka, razina postaje neprohodna. Upravo takve odluke čine srž ovog rada.

Trkački platformeri, u literaturi i praksi poznatiji pod engleskim nazivom endless runner, izdvojili su se kao zaseban podžanr krajem prvog desetljeća ovog stoljeća. Igra Canabalt iz 2009. godine [11] pokazala je da se cjelovito igračko iskustvo može izgraditi oko jedne jedine tipke: lik trči sam, a igrač odlučuje samo o trenutku skoka. Takva se postavka pokazala osobito prikladnom za dodirne zaslone, pa su je slijedile igre poput Temple Runa [12], Jetpack Joyridea [13] i Rayman Jungle Runa [14]. Jednostavnost upravljanja pritom nipošto ne znači jednostavnost izrade. Budući da igrač ne može stati niti se vratiti, svaka pogreška u razmještaju prepreka odmah postaje mjesto koje se ne da proći.

Tema rada je izrada dvodimenzionalne platformerske igre u razvojnom okruženju Unity. Igra nosi naziv „Run, Slavko, Run!”, a prati majmuna Slavka koji bježi iz zoološkog vrta kroz četiri tematski različite razine. Igra pripada podžanru trkačkih platformera, u kojem lik trči automatski, a igrač upravlja skokom i mehanikom njihanja na užetu.

Time se težište razvoja premješta s pisanja pojedine mehanike na njezino podešavanje. Mehaniku njihanja na užetu nije teško programirati, ali ju je teško postaviti tako da bude nužna, a ne ukrasna, i da igrač pritom ima stvarnu priliku pogoditi trenutak hvatanja. Isto vrijedi i za protivnika: čuvar koji nikada ne stigne igrača ne stvara nikakav pritisak, dok onaj koji ga stigne bez ijedne igračeve pogreške igru čini nepravednom. Oba se pitanja mogu razriješiti izračunom prije nego što se igra uopće pokrene, i upravo je taj postupak predmet ovog rada.

Cilj rada bio je izraditi cjelovitu igru, od izbornika do završnog ekrana, a ne samo prototip jedne mehanike. Poseban naglasak stavljen je na tri stvari:

- projektiranje fizike kretanja tako da se svaka mehanika može opravdati izračunom, a ne samo pokušajima i pogreškama;
- proceduralno sastavljanje razina iz ponavljajućih segmenata, čime se dužina razine povećava bez ručnog postavljanja svakog objekta;
- automatsku provjeru prohodnosti razine, koja za svaku prepreku provjerava je li iza nje ostalo dovoljno zaleta za sljedeću.

Doprinos rada nije u tehnološkoj novosti, budući da su sve korištene komponente standardni dio okruženja Unity, nego u sustavnom pristupu podešavanju: umjesto da se vrijednosti biraju osjećajem, u radu se pokazuje kako se iz poznatih parametara fizike izračunaju dopušteni rasponi, a zatim se dizajn razine gradi unutar tih granica.

Cjelovit izvorni kod igre, zajedno s poviješću razvoja, javno je dostupan u repozitoriju na adresi <https://github.com/ruzicjakov/run-slavko-run>.

Nakon uvoda, u drugom poglavlju opisane su korištene tehnologije. Treće poglavlje analizira žanr i postavlja zahtjeve na igru. Četvrto poglavlje, koje je i najopsežnije, opisuje izradu igre: arhitekturu programskog rješenja, fiziku kretanja, mehaniku njihanja, protivnika, sustav razina, grafiku, zvuk, korisničko sučelje i izgradnju izvršne datoteke. Peto poglavlje opisuje testiranje i podešavanje, uključujući pogreške na koje se naišlo tijekom razvoja i način na koji su otklonjene. Šesto poglavlje donosi zaključak i smjernice za daljnji rad.

2. Korištene tehnologije

2.1. Razvojno okruženje Unity

Unity je višeplatformsko razvojno okruženje za izradu igara koje se temelji na komponentnom modelu [4]. Osnovna gradivna jedinica je objekt scene (GameObject), koji sam po sebi nema ponašanje, nego ga dobiva dodavanjem komponenti. Takav pristup pogoduje ponovnoj uporabi: ista skripta za prepreku koristi se i na drvenom sanduku u zoološkom vrtu i na bačvi u tvornici, a razlikuje ih samo pridruženi sprite (sličica).

Za izradu igre korištena je verzija Unity 6000.0.40f1 uz univerzalni cjevovod iscrtavanja (Universal Render Pipeline). Popis značajnih paketa u projektu prikazan je u tablici 2.1.

Tablica 2.1. Značajni paketi u projektu (izradio autor)

Paket	Verzija	Namjena
com.unity.feature.2d	2.0.1	skup alata za 2D razvoj
com.unity.render-pipelines.universal	17.0.4	cjevovod iscrtavanja
com.unity.ugui	2.0.0	sustav korisničkog sučelja
com.unity.inputsystem	1.13.0	obrada unosa
com.unity.timeline	1.8.7	animacija u vremenskoj crti

2.2. Programski jezik C#

Ponašanje objekata u okruženju Unity opisuje se skriptama napisanima u jeziku C# [9]. Skripte nasljeđuju razred MonoBehaviour, čime dobivaju pristup metodama koje okruženje poziva automatski u određenim trenucima životnog ciklusa objekta. U radu su najviše korištene metode Awake i Start za pripremu, Update za obradu unosa jednom po slici, FixedUpdate za fizikalne izračune u stalnom vremenskom koraku, te metode odziva na sudare OnTriggerEnter2D i OnCollisionEnter2D.

Razlika između metoda Update i FixedUpdate bitna je za ovaj rad. Metoda Update poziva se jednom po iscrtanoj slici, pa njezina učestalost ovisi o brzini računala. Metoda FixedUpdate poziva se u stalnom koraku, prema zadanim postavkama svakih 0,02 sekunde, i u njoj se obavljaju svi izračuni koji djeluju na fizikalno tijelo. Sila primijenjena u metodi Update proizvela bi različit učinak na različitim računalima, što je u igri neprihvatljivo.

2.3. Fizika krutog tijela u dvije dimenzije

Fizikalno ponašanje objekata ostvaruje se komponentom Rigidbody2D [5], koja objektu daje masu, brzinu i podložnost gravitaciji, te komponentama sudarača (Collider2D), koje opisuju oblik tijela za potrebe sudara. Sudarač može raditi u dva načina. U običnom načinu tijela se fizički odbijaju, a u načinu okidača (trigger) tijela prolaze jedno kroz drugo, ali se javlja programski događaj. Prepreke u igri koriste obični način, dok predmeti koje igrač skuplja i zone opasnosti koriste način okidača.

Za povezivanje tijela koriste se zglobovi (Joint2D). U radu je korišten DistanceJoint2D [6], koji održava stalnu udaljenost između tijela i zadane točke u prostoru, čime nastaje njihalo. Izbor zgloba pokazao se ključnim i detaljno je obrazložen u poglavlju 4.4.

2.4. Sustav za praćenje izvornog koda

Tijekom razvoja korišten je sustav Git, uz mrežno spremište na servisu GitHub [10]. Rad je razvijan kroz niz zapisa promjena (commit). Promjene su grupirane tematski, tako da svaki zapis obuhvaća jednu zaokruženu izmjenu, primjerice ispravak jedne pogreške ili dodavanje jedne cjeline sučelja. Takav pristup omogućuje praćenje razvoja i vraćanje na raniju verziju ako se promjena pokaže neispravnom.

2.5. Pomoćni alati za izradu sadržaja

Sav popratni sadržaj igre, odnosno slike pozadina, pločice tla i zvučni zapisi, izrađen je programski u jeziku Python umjesto ručnim crtanjem ili snimanjem. Za slike je korištena biblioteka Pillow, a za zvuk biblioteke NumPy i standardni modul wave. Takav je pristup odabran iz dva razloga. Prvi je pravni: sadržaj nastao iz vlastitog programskog koda nema nikakvih ograničenja licencije, za razliku od preuzetih besplatnih skupova sadržaja, kod kojih je uvjete potrebno provjeravati pojedinačno. Drugi je praktični: kada se pokazalo da vidno polje kamere treba proširiti, pozadine je bilo dovoljno ponovno izgraditi promjenom jedne vrijednosti, umjesto da ih se ponovno crta.

Isti su alati korišteni i za provjeru ispravnosti. Bešavnost pozadina provjerena je usporedbom prvog i posljednjeg stupca svake slike, a bešavnost zvučnih petlji usporedbom uzoraka na početku i kraju zapisa. Obje provjere izvode se pri svakoj izgradnji sadržaja.

3. Analiza žanra i postavljanje zahtjeva

3.1. Obilježja trkačkih platformera

Trkački platformeri razlikuju se od klasičnih platformera po tome što igrač ne upravlja kretanjem naprijed. Lik trči stalnom brzinom, a igrač odlučuje samo kada će skočiti i kada će upotrijebiti posebnu mehaniku. Time se složenost upravljanja svodi na jednu ili dvije tipke, a težina se prebacuje s preciznosti pokreta na preciznost trenutka.

Iz te osnovne postavke slijedi nekoliko svojstava koja igra ovog žanra mora zadovoljiti. Prvo, igrač mora vidjeti prepreku dovoljno rano da stigne reagirati. Drugo, razmak između prepreka mora biti takav da je nakon doskoka moguće pripremiti sljedeći skok. Treće, kazna za pogrešku mora biti razmjerna, jer je u igri s automatskim kretanjem igrač lako uhvaćen u niz pogrešaka iz kojih se ne može izvući.

Teorijski okvir za projektiranje takvih razina dali su Compton i Mateas [1], koji razine platformskih igara opisuju kao hijerarhiju u četiri sloja: pojedinačne sastavnice, poput platformi, prepreka i točaka vješanja, slažu se u obrasce, obrasci u ćelije, a ćelije u razinu. Njihova je ključna tvrdnja da ritmičan razmještaj prepreka stvara ritmičan slijed pokreta igrača i time svaki pojedini skok čini lakšim za pogoditi. Smith, Cha i Whitehead [2] taj pojam razrađuju u takozvane ritmičke skupine i pokazuju da se težina dionice može opisati odnosom između brzine lika, dosega skoka i razmaka među preprekama, dakle veličinama koje se daju izračunati, a ne samo procijeniti osjećajem.

Schell [3] isti problem promatra s igračeve strane. Igra zadržava pažnju dok je težina usklađena s igračevom vještinom, pa težina mora rasti postupno, a ne skokovito. U trkačkom platformeru to znači dvoje: nove se mehanike moraju uvoditi jedna po jedna, i svaka razina treba tražiti nešto što prethodna nije. Ta je smjernica izravno oblikovala ustroj razina opisan u poglavlju 4.8.

3.2. Usporedive igre

Prije postavljanja zahtjeva pregledane su četiri igre koje se u literaturi i stručnim osvrtima redovito navode kao predstavnice žanra. Iz svake je preuzeto po jedno rješenje, a pregled je dan u tablici 3.1.

Tablica 3.1. Usporedive igre i rješenja preuzeta iz njih

Igra	Godina	Rješenje koje je preuzeto
Canabalt [11]	2009.	upravljanje jednom tipkom; kretanje naprijed je stalno, a igrač bira samo trenutak skoka
Temple Run [12]	2011.	protivnik koji stalno slijedi igrača i čini kaznu za pogrešku vidljivom, a ne samo brojanom
Jetpack Joyride [13]	2011.	predmeti koji se skupljaju i privremeni napitci, kao razlog da igrač svjesno preuzme rizik
Rayman Jungle Run [14]	2012.	svaki svijet uvodi jednu novu sposobnost, pa se mehanike uče odvojeno prije nego se kombiniraju

Rayman Jungle Run [14] bio je pritom izravan uzor za ustroj razina. Svaki od njegova četiri svijeta dodaje jednu novu sposobnost, pa igrač svaku mehaniku svlada odvojeno prije nego što ih mora

kombinirati. Ista je zamisao primijenjena i ovdje, s tom razlikom da su nove mehanike birane tako da se naslanjaju na već postojeću fiziku skoka i njihanja, a ne da uvode nove tipke.

Jedna dizajnerska odluka ne slijedi nijednu od navedenih igara, nego je vlastita: odnos prepreke i protivnika. U navedenim igrama sudar s preprekom u pravilu izravno završava vožnju ili oduzima život. U ovoj igri sudar uzrokuje samo posrtanje, a život oduzima čuvar, i to jedino ako to izgubljeno vrijeme uspije iskoristiti. Razlozi za tu odluku i njezine posljedice opisani su u poglavlju 4.6.

3.3. Zahtjevi na igru

Na temelju analize žanra postavljeni su sljedeći zahtjevi:

- igrač upravlja skokom i mehanikom hvatanja, dok se kretanje naprijed odvija samostalno;
- protivnik stalno slijedi igrača i stvara pritisak, ali ga nikada ne smije proći ili se zaglaviti u njemu;
- razine moraju sadržavati mjesta na kojima je mehanika njihanja jedini način prolaska, kako mehanika ne bi bila ukrasna;
- svaka prepreka mora biti prohodna pri brzini trčanja te razine, što se mora moći provjeriti izračunom, a ne samo igranjem;
- protivnik smije uhvatiti igrača samo kao posljedicu igračeve pogreške, a ne sam od sebe nakon dovoljno dugog trčanja, ali ga u tom slučaju mora doista moći uhvatiti;
- igra mora imati glavni izbornik, mogućnost nastavka, pauzu, zvučnu podlogu i završni ekran, odnosno biti cjelovita, a ne prototip;
- igra se mora moći izgraditi u samostalnu izvršnu datoteku koja se pokreće bez razvojnog okruženja.

Zahtjevi su tijekom rada služili i kao popis za provjeru. Svakom je zahtjevu unaprijed pridružen način na koji se njegovo ispunjenje može utvrditi, kako ocjena ne bi ovisila samo o dojmu pri igranju. Pregled je dan u tablici 3.2.

Tablica 3.2. Zahtjevi i način njihove provjere (izradio autor)

Oznaka	Zahtjev	Način provjere
Z1	kretanje naprijed je samostalno, igrač upravlja skokom i hvatanjem	igranje
Z2	čuvar slijedi igrača, ali ga ne prolazi niti se u njemu zaglavljuje	ograničenje položaja u kodu; igranje
Z3	na svakoj razini postoji	usporedba širine jame s dosegom skoka (poglavlje

Oznaka	Zahtjev	Način provjere
	mjesto na kojem je njihanje jedini prolaz	4.5)
Z4	svaka je prepreka prohodna pri brzini te razine	automatska provjera prohodnosti (poglavlje 4.7.1)
Z5	čuvar hvata igrača samo nakon igračeve pogreške, ali tada doista hvata	simulacija potjere (poglavlje 4.6)
Z6	igra je cjelovita: izbornik, nastavak, pauza, zvuk, završni zaslon	igranje; pregled scena
Z7	igra se gradi u samostalnu izvršnu datoteku	izgradnja skriptom (poglavlje 4.13)

3.4. Koncept igre

Igrač preuzima ulogu majmuna Slavka koji bježi iz zoološkog vrta. Kroz četiri razine, koje se tematski razlikuju (zoološki vrt, grad, šuma, tvornica), Slavko bježi pred čuvarom, preskače prepreke i prelazi jame njihanjem na užetu. Skupljanjem kokosa dobiva dodatne živote, napitak PowerApe omogućuje mu probijanje lomljivih prepreka, a BananaBoost ga privremeno ubrzava.

4. Izrada igre

4.1. Struktura projekta

Projekt je organiziran po vrsti sadržaja. Programski kod razdvojen je u mape prema odgovornosti, čime se izbjegava jedna velika mapa sa svim skriptama. Podjela prati odgovornosti, a ne veličinu: svaka mapa okuplja razrede koji rješavaju jedno pitanje, pa se pri izmjeni jedne mehanike gotovo nikada ne dira više od jedne mape.

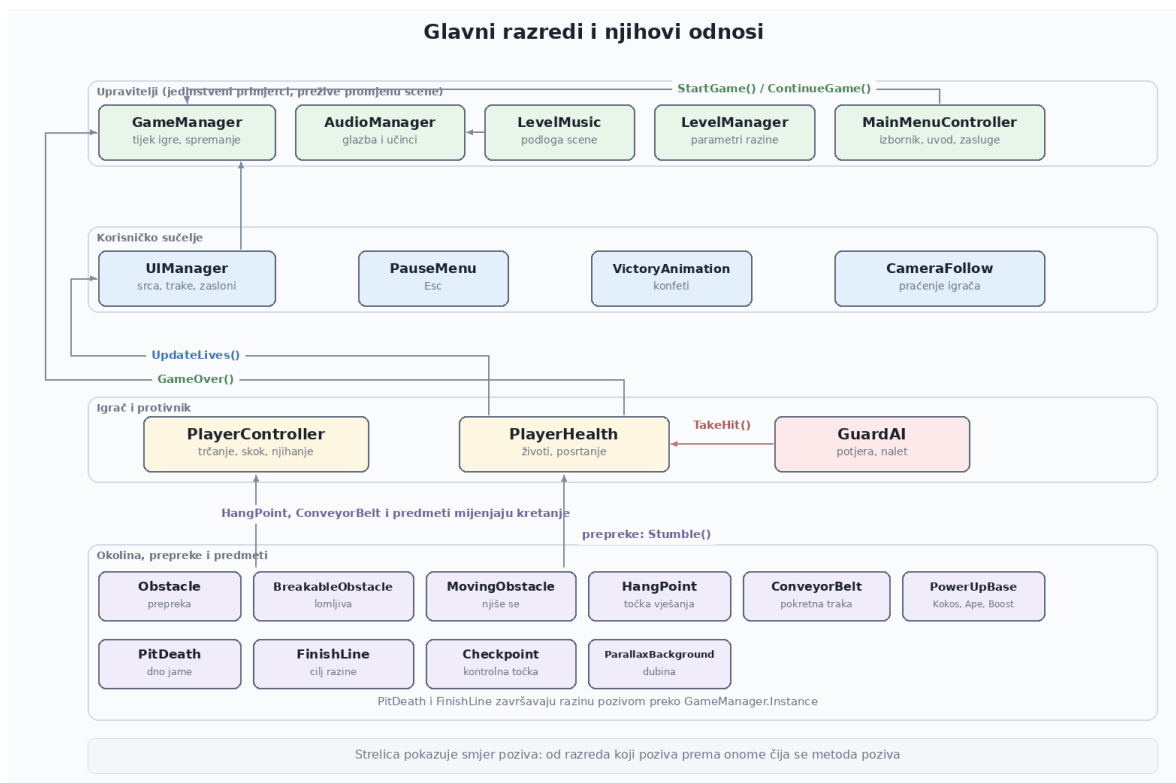
Tablica 4.1. Raspodjela programskog koda po mapama (izradio autor)

Mapa	Skripte	Sadržaj
Player	2	kretanje, skok, njihanje, životi
Managers	6	tijek igre, sučelje, zvuk, glazba razine
Obstacles	3	prepreke i točke vješanja
PowerUps	4	predmeti koje igrač skuplja
Environment	4	kontrolne točke, cilj, jame, paralaksa
UI	2	pauza i pobjednička animacija
Enemies	1	ponašanje čuvara
Camera	1	praćenje igrača
Editor	1	izgradnja izvršne datoteke

Scene su podijeljene na glavni izbornik i četiri razine. Svaka razina sadrži 101 objekt, od čega je najveći dio segmenata tla, prepreka i predmeta koje igrač skuplja, dok scena glavnog izbornika sadrži 15 objekata. Uz programski kod, projekt sadrži 38 slikovnih datoteka i 9 zvučnih zapisa.

4.2. Arhitektura programskog rješenja

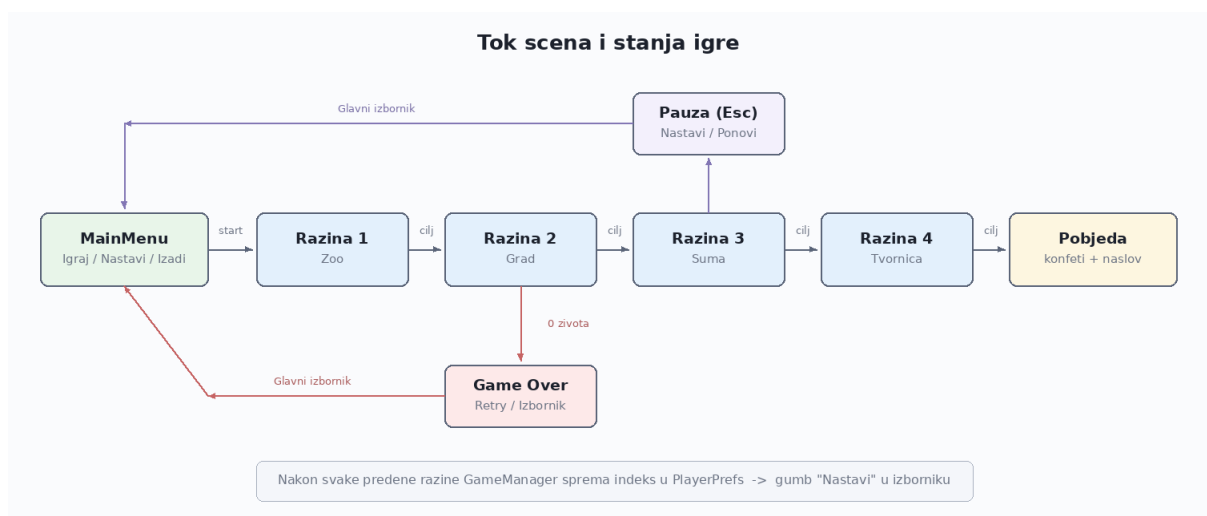
Programsko je rješenje podijeljeno u četiri sloja, prikazana na slici 4.1. Prvi čine upravitelji, izvedeni kao jedinstveni primjerci koji preživljavaju promjenu scene. Drugi je korisničko sučelje, vezano uz scenu. Treći su igrač i protivnik, a četvrti okolina, prepreke i predmeti koje igrač skuplja. Veze među slojevima idu u jednom smjeru: donji slojevi pozivaju gornje, a nikada obrnuto. Prepreka tako ne zna ništa o sučelju niti o tijeku igre, nego samo javi igraču da je posrnuo, a posljedice te obavijesti šire se prema gore.



Slika 4.1. Glavni razredi i njihovi odnosi (izradio autor)

Takva podjela odgovara komponentnom modelu okruženja Unity [4], u kojem se ponašanje ne nasljeđuje, nego sastavlja dodavanjem komponenti na objekt scene. Zbog toga se ista skripta Obstacle koristi i za drveni sanduk u zoološkom vrtu i za bačvu u tvornici, a razlikuju ih samo pridruženi sprite i položaj.

Tijek igre vodi razred GameManager, izveden kao jedinstveni primjerak (singleton) koji preživljava promjenu scene. On pamti redni broj trenutne razine, pokreće učitavanje sljedeće i upravlja stanjima završetka razine, poraza i pobjede. Sučeljem upravlja razred UIManager, koji je vezan uz scenu i prikazuje broj života, trajanje napitka i završne zaslone. Tijek scena i prijelazi među stanjima igre prikazani su na slici 4.2.



Slika 4.2. Tok scena i stanja igre (izradio autor)

Napredak se sprema u trajnu pohranu (PlayerPrefs) nakon svake prijedene razine, čime gumb „Nastavi” u izborniku dobiva smisao. Nakon prelaska cijele igre zapis se briše, jer nastavak više nema smisla.

Podjela na trajne i scenske komponente nije stvar ukusa. Upravitelji moraju preživjeti promjenu scene jer čuvaju stanje koje traje dulje od jedne razine, primjerice redni broj razine ili odabranu glasnoću. Sučelje, nasuprot tome, mora nestati zajedno sa scenom, jer se svaki njegov element odnosi na objekte upravo te scene. Miješanje tih dviju uloga bilo je uzrok jedne od pogrešaka navedenih u tablici 5.1: gumbi izbornika prestali su raditi jer su upućivali na komponentu koja se pri ponovnom učitavanju scene uništava.

Razredi međusobno komuniciraju izravnim pozivima metoda, bez posebnog sustava događaja. Za projekt ove veličine to je jednostavnije za praćenje, jer se iz slike 4.1. u svakom trenutku vidi tko koga poziva. Cijena takvog izbora je čvršća vezanost razreda, koja bi u znatno većem projektu postala smetnja.

4.3. Kretanje igrača

Igrač se kreće automatski, tako da se u svakoj slici vodoravna komponenta brzine postavlja na zadanu vrijednost, dok okomita ostaje pod utjecajem gravitacije:

```
rb.linearVelocity = new Vector2(runSpeed * speedMultiplier,  
                                rb.linearVelocity.y);
```

Skok se izvodi trenutnim impulsom prema gore, ali samo ako je lik na tlu. Provjera dodira s tlom obavlja se kružnim ispitivanjem preklapanja oko pomoćne točke smještene uz noge lika, čime se izbjegava oslanjanje na događaje sudara koji ne razlikuju tlo od bočnog dodira s preprekom.

Parametri kretanja i izvedene veličine prikazani su u tablici 4.2. Vrijednost efektivne gravitacije dobivena je množenjem osnovne gravitacije okruženja iznosa $9,81 \text{ m/s}^2$ s faktorom gravitacije tijela, koji iznosi 3.

Tablica 4.2. Parametri kretanja i izvedene veličine (izradio autor)

Veličina	Vrijednost	Način dobivanja
Brzina trčanja (razina 1)	6 j/s	zadano
Sila skoka	9 j/s	zadano
Faktor gravitacije	3	zadano
Efektivna gravitacija	$29,43 \text{ j/s}^2$	$3 \times 9,81$
Vrijeme do najviše točke	0,306 s	v_0 / g
Visina skoka	1,376 j	$v_0^2 / (2g)$
Trajanje skoka	0,612 s	$2v_0 / g$
Vodoravni doseg skoka	3,67 j	$v \times t$

Oznaka „j” označava Unityjevu jedinicu duljine, koja u ovom projektu odgovara stotini piksela izvorne slike.

4.4. Mehanika njihanja na užetu

Mehanika njihanja središnji je dio rada i prošla je kroz značajnu izmjenu tijekom razvoja. U prvoj izvedbi korišten je zglobov HingeJoint2D uz sidrište postavljeno u središte tijela igrača. Takva postavka ne stvara uže, nego izravno spaja središte lika s točkom vješanja, pa se lik pri hvatanju trenutno premjestio na granu i tamo ostao. Mehanika je zapravo bila neupotrebljiva.

Rješenje je bila zamjena zglobov DistanceJoint2D, koji održava stalnu udaljenost između tijela i točke vješanja i time doslovno predstavlja uže zadane duljine:

```
ropeJoint = gameObject.AddComponent<DistanceJoint2D>();
ropeJoint.autoConfigureDistance = false;
ropeJoint.autoConfigureConnectedAnchor = false;
ropeJoint.connectedBody = null;
ropeJoint.connectedAnchor = anchor;
ropeJoint.anchor = Vector2.zero;
ropeJoint.distance = Mathf.Clamp(currentDistance,
                                minRopeLength, maxRopeLength);
```

Duljina užeta ne postavlja se na unaprijed zadanu vrijednost, nego na stvarnu udaljenost u trenutku hvatanja, ograničenu na raspon od 1,2 do 3 jedinice. Time se izbjegava nagli trzaj koji bi nastao kada bi zglobov morao odmah ispraviti razliku između stvarne i zadane duljine.

4.4.1. Prikaz užeta

Uže se iscrtava komponentom LineRenderer. Tijekom razvoja pokazalo se da se ta komponenta ne može dodati na objekt igrača, jer okruženje Unity dopušta samo jedan iscrtavač po objektu, a objekt igrača već ima komponentu SpriteRenderer. Poziv za dodavanje tiho ne uspijeva, pa se uže uopće nije prikazivalo. Rješenje je stvaranje zasebnog objekta djeteta koji nosi iscrtavač linije i koji se stvara pri hvatanju, a uklanja pri otpuštanju.

4.4.2. Pomoć igraču

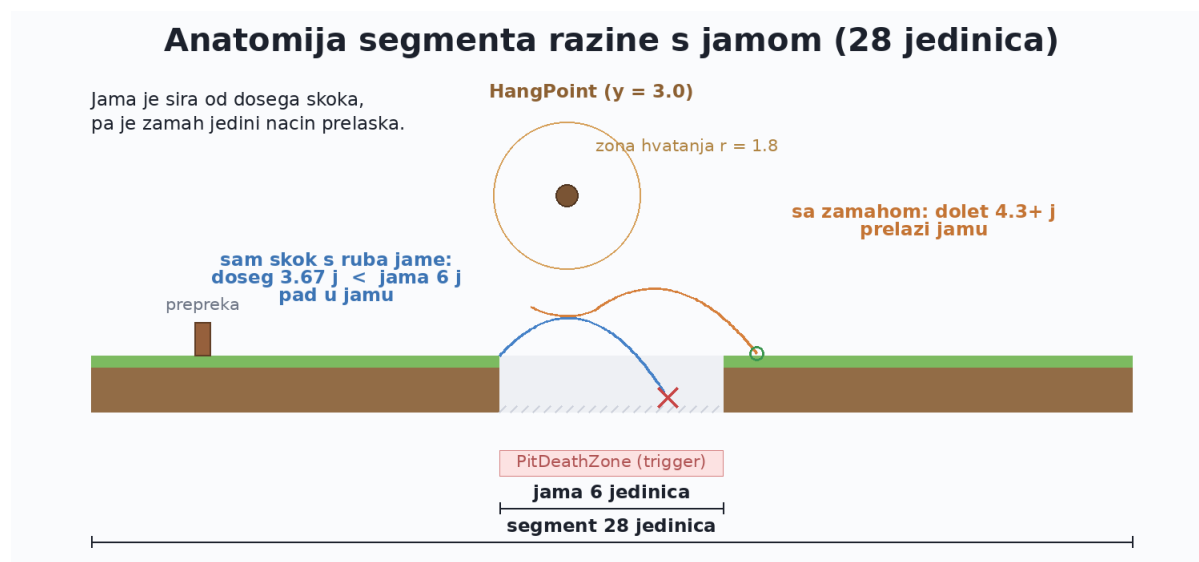
Njihalo prepušteno samo sebi gubi zamah, pa je uvedena stalna sila prema naprijed iznosa 14 N dok lik visi. Otpuštanje se pretvara u izbačaj, tako da se vodoravna brzina uzima kao veća vrijednost između trenutne i brzine trčanja, uvećana za 15 posto, dok se okomitoj komponenti osigurava iznos od najmanje 7 jedinica:

```
float vx = Mathf.Max(Mathf.Abs(v.x), runSpeed * speedMultiplier)
            * swingReleaseBoost;
float vy = Mathf.Max(v.y, releaseUpwardBoost);
rb.linearVelocity = new Vector2(vx, vy);
```

Dodane su i dvije mjere koje smanjuju kažnjavanje neprecizne igre: zona hvatanja proširena je na polumjer od 1,8 jedinica, a uvedeno je i razdoblje od 0,15 sekundi nakon izlaska iz zone tijekom kojeg se lik još uvijek može uhvatiti. Bez toga je trebalo pogoditi točku gotovo u piksel.

4.5. Projektiranje jama

Kako bi mehanika njihanja imala stvarnu svrhu, jame su namjerno projektirane šire od dosega skoka. Iz tablice 4.2. slijedi da vodoravni doseg skoka pri brzini 6 iznosi 3,67 jedinica, a pri najvećoj brzini u igri, koja iznosi 7,5, doseg je 4,59 jedinica. Odabrane su dvije širine jama, 6 i 7,5 jedinica, čime je i u najpovoljnijem slučaju osigurano da sam skok nije dovoljan. Građa takvog segmenta razine prikazana je na slici 4.3.



Slika 4.3. Anatomija segmenta razine s jamom (izradio autor)

Položaj točke vješanja također je izračunat, a ne pogođen. U ranijoj inačici točka je stajala u sredini jame na visini 2,5 jedinica. Provjerom se pokazalo da je takva točka nedostižna: dok lik s ruba jame prijeđe tri jedinice do sredine, već se spušta, pa vrh njegova sudarača dosegne visinu 1,71 jedinicu, dok je donji rub zone hvatanja na 1,90. Razlika od 0,19 jedinica znači da hvatanje nikada nije bilo moguće.

U ispravljenoj izvedbi točka vješanja pomaknuta je na 1,2 jedinice prije sredine jame i podignuta na visinu 3 jedinice, uz zonu hvatanja polumjera 1,8. Time prozor za hvatanje traje 0,54 sekunde, odnosno 3,23 jedinice vodoravnog puta, a donji rub zone ostaje na visini 1,2 jedinice, što je i dalje iznad vrha lika koji stoji na tlu (0,89), pa se uže ne može uhvatiti bez skoka.

Prohodnost je zatim provjerena numeričkom simulacijom njihala uz stalni vremenski korak od 0,02 sekunde, jednak koraku fizike u okruženju. Za svaku kombinaciju razine i trenutka hvatanja izračunat je udio trajanja njihaja u kojem otpuštanje vodi do uspješnog prelaska. Rezultati su prikazani u tablici 4.3.

Tablica 4.3. Udio uspješnih trenutaka otpuštanja (izradio autor)

Razina	Brzina	Široka jama	Uspješan prozor
1 – Zoo	6,0	7,5 j	59 – 72 %
2 – Grad	6,5	7,5 j	61 – 73 %
3 – Šuma	7,0	11 j (lančana)	69 %
4 – Tvornica	7,5	7,5 j	65 – 80 %

Vrijednosti pokazuju da je prelazak moguć u najvećem dijelu njihaja, ali ne i trenutnim otpuštanjem pri hvatanju, čime se od igrača traži da se doista zanjiše.

Vrijednost za treću razinu odnosi se na lančanu jamu opisanu u nastavku, i to na prvo otpuštanje: u 69 % trenutaka otpuštanja s prve točke igrač doseže zonu druge točke. Za ostale se razine vrijednost odnosi na običnu široku jamu.

Na trećoj razini ista je mehanika proširena u lančano njihanje. Jama je ondje široka 11 jedinica i ima dvije točke vješanja umjesto jedne, postavljene 1,5 i 6 jedinica od lijevog ruba, obje na visini 3 jedinice. Širina je odabrana izračunom: simulacija njihala pokazuje da jedan njihaj s prve točke nosi lika 9,57 jedinica, dok dva povezana njihaja nose 14,84 jedinice. Jama od 11 jedinica time pada između tih dviju vrijednosti, pa se ne može prijeći jednim hvatanjem, ali se pouzdano prelazi ako igrač u letu uhvati i drugo užje.

4.6. Protivnik

Čuvar koristi jednostavan model približavanja s elastičnom vezom: ako je igrač udaljeniji od zadane granice, čuvar se kreće povećanom brzinom, inače osnovnom. Osnovna brzina namjerno je manja od brzine trčanja igrača, pa čuvar u ispravnoj vožnji polako zaostaje. Brzine rastu po razinama zajedno s brzinom igrača, tako da odnos snaga ostaje isti, a raste samo zahtjev na brzinu reakcije. Vrijednosti po razinama navedene su u tablici 4.4.

Tablica 4.4. Brzine igrača i čuvara po razinama (izradio autor)

Razina	Igrač	Čuvar (osnovna)	Čuvar (sustizanje)	Granica
1 – Zoo	6,0	5,6	7,5	3,0 j
2 – Grad	6,5	6,1	8,0	3,0 j
3 – Šuma	7,0	6,6	8,5	3,0 j
4 – Tvornica	7,5	7,1	9,0	3,0 j

Tijekom razvoja pojavila se pogreška u kojoj je čuvar prolazio kroz igrača. Uzrok je bio oslanjanje na događaj ulaska u okidač, koji se javlja samo jednom. Kada je igrač bio zaustavljen ispred prepreke, čuvar bi ga dotaknuo jednom, nakratko stao, a zatim nastavio kretanje i prošao kroz njega. Rješenje je tvrdo ograničenje položaja u metodi FixedUpdate:

```
float limitX = player.position.x - stopDistance;

if (transform.position.x >= limitX)
{
    rb.linearVelocity = new Vector2(0f, rb.linearVelocity.y);
    if (transform.position.x > limitX)
        transform.position = new Vector3(limitX,
            transform.position.y, transform.position.z);
    return;
}
```

Takvo rješenje ne ovisi o događajima sudara, pa nema stanja u kojem bi ograničenje moglo zakazati.

Nakon te ispravke pojavio se suprotan problem: čuvar više nikada nije uspijevao uhvatiti igrača. Uzrok je bio u načinu na koji se hvatanje provjeravalo. Čuvar je pokušavao pogoditi igrača u odzivu na sudar okidača, koji se javlja samo dok se sudarači doista preklapaju. Kada bi igrač primio udarac, postao bi nepovrediv na 1,5 sekundu, a upravo bi u tom razdoblju čuvar bio najbliže. Do isteka nepovredivosti čuvar bi se već odmaknuo dovoljno da se sudarači više ne dodiruju, pa poziv za pogodak nikada ne bi bio ponovljen. Ispravak je provjera hvatanja po udaljenosti, koja se izvodi u svakom koraku fizike:

```
if (distance <= reachDistance &&
    Mathf.Abs(player.position.y - transform.position.y)
    <= reachHeight)
{
    TryHit();
}
```

Uvjet na razliku visina potreban je zato što bi inače čuvar mogao zgrabiti igrača koji visi na užetu iznad njega.

4.6.1. Prepreke kao izvor pritiska umjesto izravne štete

Preostao je i drugi problem, koji se nije mogao otkloniti samo brojkama. Osnovna brzina čuvara manja je od igračeve, pa ga čuvar u ispravnoj vožnji nikada ne sustiže. Ako se čuvar ubrza toliko da sam sustigne igrača, prestaje biti posljedica igračeve pogreške i postaje neizbježan; ako se ne ubrza, nikada nije opasan. Rješenje je stoga bilo u dizajnu, a ne u podešavanju: čuvaru je trebao trenutak u kojem igrač izgubi tlo.

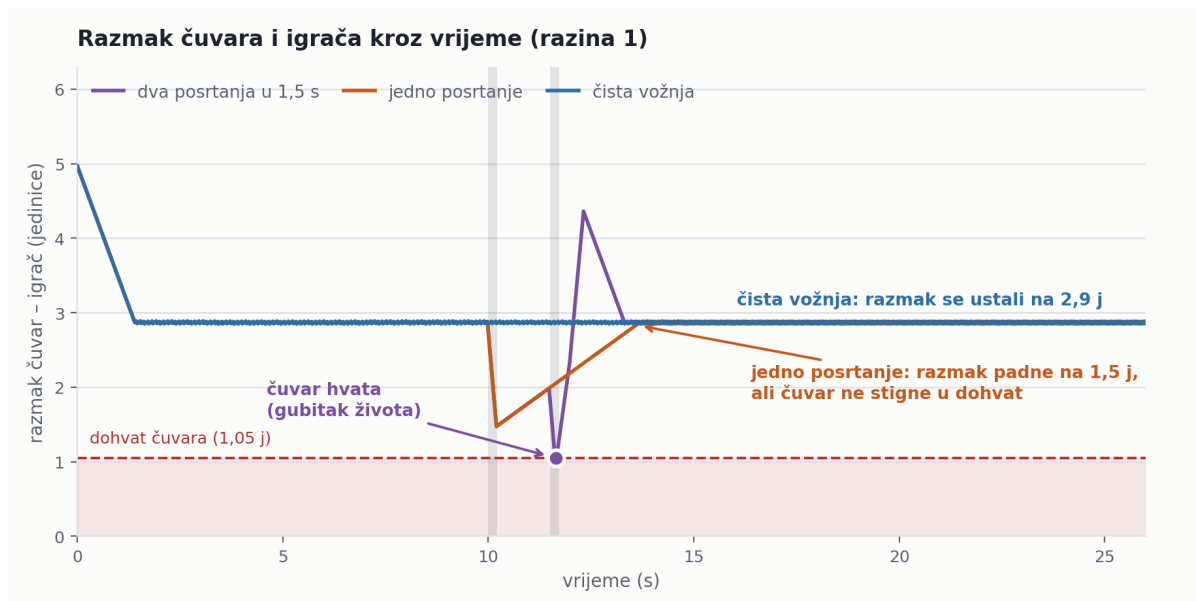
U ranijoj izvedbi sudar s preprekom oduzimao je život izravno, a prepreka bi odmah nestala kako igrač ne bi ostao zaglavljen. Time je pogreška bila kažnjena, ali nije trajala, pa čuvar nije imao priliku prići. Sudar s preprekom stoga je pretvoren u posrtanje, koje ne oduzima život, nego oduzima vrijeme:

```
public bool Stumble()
{
    if (isInvulnerable) return false;
    if (Time.time < stumbleCooldownUntil) return false;

    stumbleCooldownUntil = Time.time + stumbleCooldown;
    rb.linearVelocity = new Vector2(-stumbleNudge,
                                    rb.linearVelocity.y);
    playerController?.NotifyKnockback(stumbleStopDuration);
    return true;
}
```

Brzina se namjerno postavlja izravno, a ne impulsom, kako bi gubitak tla bio jednak bez obzira na to kojom je brzinom igrač udario u prepreku. Život sada oduzima isključivo čuvar, a jame i dalje znače trenutni kraj razine. Uloge su time razdvojene: prepreka stvara priliku, a čuvar je iskorištava.

Ponašanje je zatim provjereno simulacijom za nekoliko obrazaca igranja. Rezultati su prikazani na slici 4.4. i u tablici 4.5.



Slika 4.4. Razmak čuvara i igrača kroz vrijeme, za tri načina igranja (izradio autor)

Tablica 4.5. Ishod simulacije potjere za različite obrasce pogrešaka (izradio autor)

Način igranja	Najmanji razmak	Hvatanja
čista vožnja	2,85 j	0
jedno posrtanje	1,47 j	0
dva posrtanja u razmaku 1,5 s	0,96 j	1
dva posrtanja u razmaku 3,0 s	1,18 j	0
dva posrtanja u razmaku 5,0 s	1,45 j	0

Dobiveno ponašanje odgovara traženom: čista vožnja nikada ne završava hvatanjem, jedna pogreška se oprašta, a dvije pogreške unutar približno tri sekunde znače gubitak života. Razmak se nakon posrtanja vraća na sigurnu vrijednost za 2,6 sekundi, što pri brzini 6 odgovara približno 16 jedinica staze, dakle nešto više od polovine segmenta. Budući da je najmanji razmak između dviju prepreka 7,25 jedinica, dvije uzastopne pogreške doista padaju unutar tog prozora.

4.6.2. Odbacivanje nakon udarca

Pri ranijim pokušajima da se čuvar učini opasnijim samo povećanjem brzine, ishod je bio neočekivan: u jednoj vožnji zabilježena su 34 pogotka, iako igrač nije radio više pogrešaka nego prije. Uzrok je bio u smjeru odbacivanja.

Razred PlayerHealth odbacivao je igrača uvijek prema natrag, jer je odbacivanje izvorno bilo zamišljeno za sudar s preprekom koja se nalazi ispred igrača. Kada je pogodak dolazio od čuvara, koji se nalazi iza igrača, odbacivanje je guralo igrača ravno u čuvara, pa bi nakon isteka nepovredivosti odmah uslijedio novi pogodak. Rješenje je bilo uvođenje smjera odbacivanja kao ulazne veličine, tako da izvor pogotka sam određuje kamo će igrač biti odbačen:

```
public bool TakeHit(float pushDirX = -1f)
{
```

```

    if (isInvulnerable) return false;
    ...
    float dir = pushDirX >= 0f ? 1f : -1f;
    rb.AddForce(new Vector2(dir * knockbackForce,
                           knockbackForce * 0.5f),
               ForceMode2D.Impulse);

```

Uz to je čuvaru dodano razdoblje mirovanja od 0,7 sekundi nakon ostvarenog pogotka. Zajedno s nepovredivošću igrača, to daje približno 1,6 sekundi tijekom kojih se razmak vrati na 3,5 jedinice, pa se drugi pogodak ne može dogoditi prije nego što igrač povрати nadzor.

4.6.3. Nalet na posljednjoj razini

Na četvrtoj razini čuvar dobiva još jedno ponašanje. Svaki devet sekundi kreće u nalet: brzina mu se na 0,9 sekundi podiže na 8,6 jedinica u sekundi, što je iznad igračeve brzine trčanja od 7,5. Nalet sam po sebi ne stiže igrača koji trči čisto, nego mu skraćuje zalihu razmaka, pa pogreška napravljena neposredno nakon naleta stoji više nego ista pogreška u mirnijem trenutku.

To se ponašanje moralo uskladiti s pokretnim trakama, koje na istoj razini usporavaju igrača na 0,85 njegove brzine. Prva izvedba ta dva pravila nije povezivala, pa se dogodilo da nalet zatekne igrača dok stoji na usporavajućoj traci. U simulaciji čiste vožnje takav je spoj dao šest hvatanja, iako igrač nije napravio nijednu pogrešku. Kazna tada nije dolazila od igračeve odluke, nego od poklapanja dvaju mjerača vremena.

Prvi pokušaj ispravka samo je odgađao nalet dok igrač ne siđe s trake, ali je ishod ostao isti, jer bi se odgođeni nalet pokrenuo neposredno nakon silaska. Konačno rješenje nalet u toj situaciji poništava i zakazuje sljedeću provjeru za pola sekunde:

```

bool slowed = playerController != null &&
              playerController.EnvironmentSpeedFactor < 0.95f;
if (slowed) { burstUntil = 0f; }

```

Uz to su trake razmaknute tako da između dviju usporavajućih dionica ostane najmanje 42 jedinice staze. Nakon te izmjene čista vožnja četvrtom razinom u simulaciji ne završava nijednim hvatanjem, dok dvije pogreške u kratkom razmaku i dalje znače gubitak života.

4.7. Sastavljanje razina

Razine se ne slažu ručno, nego se sastavljaju iz segmenata širine 28 jedinica. Šest vrsta segmenata izmjenjuje se u nizu: skok preko prepreke, uska jama s njihanjem, lomljiva prepreka s napitkom, široka jama s njihanjem, bonus dionica s kokosima i dionica za predah. Svaka razina sastoji se od 14 segmenata, čime ukupna duljina staze iznosi 392 jedinice, uz dodatnu dionicu s ciljem. Raspodjela programskog koda po mapama prikazana je u tablici 4.1.

Takav pristup omogućio je da se sadržaj razine mijenja izmjenom pravila, a ne pomicanjem stotina objekata. Primjerice, povećanje broja prepreka sa sedam na dvadeset i jednu po razini izvedeno je izmjenom rasporeda unutar segmenata.

4.7.1. Provjera prohodnosti

Uz sastavljanje razine razvijena je i provjera koja za svaku prepreku računa mjesto doskoka i zalet preostao do sljedeće opasnosti, pri brzini te razine. Provjera razlikuje sljedeću prepreku od sljedeće jame, jer jama zahtijeva dulji zalet. Rezultati provjere za konačnu inačicu prikazani su u tablici 4.6.

Tablica 4.6. Rezultat provjere prohodnosti (izradio autor)

Razina	Prepreke	Jame	Viseće i trake	Najmanji razmak
1 – Zoo	21	5	–	7,25 j
2 – Grad	16	5	5 visećih	7,00 j
3 – Šuma	19	5	7 visećih	6,50 j
4 – Tvornica	19	5	9 traka	4,00 j

Provjera se izvodi pri svakom sastavljanju razina, pa promjena koja bi razinu učinila neprohodnom biva otkrivena odmah, bez potrebe za igranjem cijele razine.

Iz tablice se vidi i koliko se razine razlikuju gustoćom. Prosječan razmak između dviju opasnosti pada s 13,60 jedinica na prvoj razini na 12,76 na drugoj, 10,00 na trećoj i 9,06 na četvrtoj, dok najmanji razmak pada sa 7,25 na 4,00 jedinice. Budući da istodobno raste i brzina trčanja, vrijeme kojim igrač raspolaže između dviju odluka na četvrtoj je razini manje od polovice onoga na prvoj.

Provjera je tijekom rada proširena još jednim pravilom. Viseća prepreka ne preskače se, nego se ispod nje proturči, pa ona ne smije stajati unutar luka skoka koji igrač izvodi zbog prethodne prepreke; u protivnom nastaje sudar koji se ne može izbjeći. Provjera zato za svaku viseću prepreku računa raspon u kojem se igrač može naći u zraku i odbacuje raspored u kojem se ta dva raspona preklapaju.

4.7.2. Programsko stvaranje scena

Segmenti se ne slažu u uređivaču scena, nego se datoteka scene ispisuje programski. Razlog je veličina: jedna razina sadrži oko petsto objekata, pa bi njihovo ručno postavljanje bilo dugotrajno, a svaka izmjena pravila značila bi ponavljanje istog posla na sve četiri razine. Umjesto toga, pomoćna skripta piše datoteku u istom zapisu koji koristi i samo okruženje, pa se dobivena scena otvara i uređuje kao i svaka druga.

Zapis scene je tekstualan i sastoji se od blokova, od kojih svaki opisuje jedan objekt ili jednu komponentu. Svaki blok ima vlastitu oznaku, a veze među njima nisu imena, nego upravo te oznake:

```
--- !u!1 &1836420001
GameObject:
  m_Name: Obstacle
  m_Component:
  - component: {fileID: 1836420002}
```

Takav zapis je moćan, ali i osjetljiv: pogrešna ili ponovljena oznaka ne prijavljuje se kao pogreška, nego se očituje kao objekt koji u igri nedostaje ili nema komponentu. Zato se svaka izrađena scena odmah i provjerava. Provjera potvrđuje da je svaka oznaka jedinstvena, da se svaka veza razrješuje u postojeći objekt ili u poznati vanjski zapis, da svaka komponenta upućuje natrag na objekt kojemu pripada te da popis korijenskih objekata scene sadrži točno one objekte koji nemaju roditelja.

Uz to se prije svake izmjene skripte najprije potvrđuje da ona zatečene scene proizvodi znak po znak jednake postojećima. Tek kada je to potvrđeno, izmjena se unosi, pa se razlika u ispisu može pripisati isključivo toj izmjeni. Time je izbjegnuta najopasnija pogreška ovakvog pristupa, a to je da izmjena namijenjena jednoj razini neopazice promijeni nešto drugo u svima.

4.8. Razlike među razinama

Četiri razine dijele isti sustav segmenata, ali se ne razlikuju samo temom i brzinom. Da bi svaka razina imala razlog za postojanje, primijenjeno je načelo preuzeto iz igre Rayman Jungle Run [14] i opisano u poglavlju 3.2: svaka razina zadržava sve mehanike prethodnih i dodaje barem jednu novu, koja se prvo pojavljuje sama, a tek na sljedećoj razini u spoju s ostalima. Pregled je dan u tablici 4.7.

Tablica 4.7. Sadržaj razina (izradio autor)

Razina	Brzina (j/s)	Nova mehanika	Čemu služi
1. Zoo	6,0	skok, njihanje, uske i široke jame	uvodi osnove upravljanja
2. Grad	6,5	višeća prepreka; pokretna prepreka (2 j / 2,5 s)	traži i zadržavanje skoka, a ne samo skok
3. Šuma	7,0	lančano njihanje (jama 11 j, dvije točke); brža pokretna prepreka (2,5 j / 2 s)	spaja mehanike druge razine i sužava prozore
4. Tvornica	7,5	pokretne trake (0,85 i 1,25); nalet čuvara (8,6 j/s svakih 9 s)	oduzima zalihu razmaka, pa se prva pogreška ne oprašta

Prva razina uvodi osnove: skok preko prepreke, njihanje na užetu te uske i široke jame. Druga razina dodaje dvije prepreke koje mijenjaju odnos prema skoku. Višeća prepreka stoji na visini skoka, pa se ispod nje mora protrčati; ona je prva prepreka u igri koju igrač rješava time što ne skoči. Pokretna prepreka niže se vodoravno s amplitudom od 2 jedinice i ciklusom od 2,5 sekunde, čime uvodi odabir trenutka skoka.

Treća razina spaja te dvije mehanike i pooštrava ih. Pokretne prepreke ondje imaju veću amplitudu, 2,5 jedinice, i kraći ciklus od 2 sekunde, pa je prozor za prolaz uži. Uz to se pojavljuje lančano njihanje opisano u poglavlju 4.5, koje traži dva hvatanja u nizu. Četvrta razina dodaje pokretne trake, koje mijenjaju brzinu igrača na 0,85 odnosno 1,25 njegove uobičajene, i nalet čuvara opisan u poglavlju 4.6.3.

Uz mehanike se razlikuje i izgled prepreka. Iako sve razine koriste istu skriptu Obstacle, svakoj je temi pridružen vlastiti sprite, pa je prepreka u zoološkom vrtu drveni sanduk, u gradu betonski blok, u šumi panj, a u tvornici bačva. To je bila i jedna od uočenih pogrešaka: u ranijoj izvedbi sastavljač razina svim je temama pridjeljivao sprite prve razine, pa su razine izgledale jednako iako su se sadržajem razlikovale.

Odnos brzine i sadržaja prikazan je na slici 4.5.



Slika 4.5. Što svaka razina donosi novo (izradio autor)

4.9. Predmeti koji se skupljaju i kontrolne točke

Uz prepreke, razine sadrže i predmete koje igrač skuplja. Sva tri dijele zajednički razred PowerUpBase, koji obavlja ono što im je svima jednako: prepoznaje dodir igrača preko okidača, pusti zvuk, prepusti djelovanje izvedenom razredu i ukloni predmet sa scene. Izvedeni razredi opisuju samo učinak, pa je svaki od njih kratak.

Kokos dodaje jedan život, do najviše tri, i odmah osvježava prikaz srca u sučelju. Napitak PowerApe pet sekundi omogućuje probijanje lomljivih prepreka umjesto zaobilaženja. Banana pet sekundi množi brzinu trčanja s 1,6, pa je to jedini trenutak u igri u kojem igrač ne samo da zadržava razmak pred čuvarom, nego ga i povećava.

Predmeti se ne postavljaju ručno, nego ih sastavljač razina smješta prema vrsti segmenta, tako da svaki stoji ondje gdje njegov učinak ima smisla. PowerApe se pojavljuje neposredno prije lomljive prepreke, kokos u bonus dionici, a banana u dionici za predah, gdje ubrzanje ne dovodi igrača ravno u sljedeću prepreku. Time predmet nije samo nagrada, nego i najava onoga što slijedi.

Duž razine postavljene su i kontrolne točke. Razred Checkpoint pamti položaj posljednje prijedene točke, a razred LevelManager pri pokretanju scene, ako takav položaj postoji, premješta igrača na njega umjesto na početak razine. Ponovni pokušaj nakon gubitka svih života time ne znači ponavljanje cijele razine.

Položaj kontrolne točke čuva se u statičkom polju, jer mora preživjeti ponovno učitavanje scene. Upravo zato ga je bilo nužno izričito brisati pri prelasku na sljedeću razinu i pri povratku u glavni izbornik: bez toga bi se igrač na novoj razini zatekao na položaju zapamćenom u prethodnoj, što je pogreška koja se ne bi prijavila ni na koji način osim čudnim početkom razine.

4.10. Grafika i pozadine

Grafika je izrađena u plošnom stilu, bez sjenčanja i tekture, čime se postiže ujednačen izgled uz malen broj datoteka. Pozadina svake razine sastavljena je od tri sloja koji se pomiču različitim brzinama, čime nastaje dojam dubine. Faktori pomicanja iznose 0,15 za najdalji sloj, 0,35 za srednji i 0,6 za najbliži. Slojevi pozadine prve razine prikazani su na slici 4.6.



Slika 4.6. Slojevi pozadine razine Zoo, prikazani jedan do drugog radi provjere bešavnosti spoja (izradio autor)

Sve pozadine i tlo izrađeni su tako da se mogu bešavno ponavljati po vodoravnoj osi. Bešavnost je postignuta uporabom periodičkih funkcija čiji period stane cijeli broj puta u širinu slike, te crtanjem elemenata koji prelaze rub i u pomaknutim kopijama. Ispravnost spoja provjerena je programski, usporedbom prvog i posljednjeg stupca svake slike.

4.11. Zvuk

Zvučna podloga izrađena je programski, postupkom opisanim u poglavlju 2.5. Riječ je o sintetiziranoj glazbi u stilu ranih računalnih igara (engl. chiptune), sastavljenoj od triju glasova: pravokutnog vala za melodiju, trokutastog za bas i kratkih naleta šuma za udaraljke. Svaka je skladba građena iz osam taktova s ponavljajućim slijedom akorda, a rez zapisa postavljen je točno na kraj posljednjeg takta, čime petlja nema čujnog spoja.

Izrađeno je pet glazbenih petlji i četiri zvučna učinka, ukupno devet zapisa u obliku 16-bitnog jednokanalnog zapisa pri 44,1 kHz. Ugođaj svake razine određen je tempom i tonskim rodом, kako je prikazano u tablici 4.8.

Tablica 4.8. Glazbene petlje po scenama (izradio autor)

Zapis	Scena	Tempo	Ugođaj
music_menu	Glavni izbornik	104	C-dur, mirno
music_zoo	Razina 1 – Zoo	132	C-dur, poskočno
music_city	Razina 2 – Grad	146	a-mol, uporno
music_forest	Razina 3 – Šuma	118	e-mol, tajanstveno

Zapis	Scena	Tempo	Ugođaj
music_factory	Razina 4 – Tvornica	160	d-mol, oštro

Reprodukcijom upravlja razred AudioManager, izveden kao jedinstveni primjerak koji preživljava promjenu scene. Izvori zvuka (AudioSource) [8] ne postavljaju se u sceni, nego ih razred stvara sam pri pokretanju, čime se izbjegava ručno podešavanje u svakoj od pet scena:

```
if (musicSource == null)
{
    musicSource = gameObject.AddComponent<AudioSource>();
    musicSource.playOnAwake = false;
    musicSource.loop = true;
}
```

Koja se skladba svira, određuje scena, a ne upravitelj. Svaka razina nosi malen razred LevelMusic koji u sebi drži zapis te razine i pri pokretanju ga predaje upravitelju. Time upravitelj ne mora poznavati popis razina, pa dodavanje nove razine ne zahtijeva izmjenu njegova koda.

Pri izradi se pokazalo da glazba prekida i ponovno počinje pri svakom ponovnom pokretanju razine, što je nakon nekoliko pokušaja postajalo zamorno. Uzrok je bio u tome što se pri učitavanju scene traži reprodukcija istog zapisa koji već svira. Rješenje je provjera prije pokretanja, tako da se zahtjev za zapisom koji već svira zanemaruje:

```
if (musicSource.clip == clip && musicSource.isPlaying) return;
```

Glasnoća glazbe postavljena je na 0,35, a glasnoća zvučnih učinaka na 0,7, kako podloga ne bi nadglasala povratnu informaciju o skoku, pogotku i skupljanju predmeta.

4.12. Korisničko sučelje

Sučelje je izgrađeno sustavom uGUI, uz platno postavljeno komponentom Canvas Scaler [7] na način prilagodbe veličini zaslona s referentnom razlučivosti 1920 × 1080. Ta je postavka bila izvor pogreške tijekom razvoja: platno je prvotno bilo postavljeno na stalnu veličinu u pikselima uz referencu 800 × 600, pa se pri punoj razlučivosti sučelje nije prilagođavalo, nego su elementi ostajali sitni i nepravilno raspoređeni.

Broj života prikazuje se trima srcima u gornjem lijevom kutu, dok se trajanje napitka prikazuje trakom u gornjem desnom kutu, koja se prazni i mijenja boju ovisno o vrsti napitka. Prikaz preostalih života vidljiv je na slici 4.7.

Budući da se razina sastoji od ponavljajućih segmenata, igrač teško procjenjuje koliko je staze ostalo. Zato je na kraj svake razine postavljen vidljiv cilj. Ranije je razina završavala nevidljivim okidačem, pa je prijelaz na sljedeću razinu djelovao kao da se dogodio sam od sebe. Sada staza završava vratima s natpisom, koja se vide prije nego što se do njih stigne, pa igrač zna da je pred krajem.

Glavni izbornik proširen je dvama zaslonima. Gumb za igru više ne pokreće razinu izravno, nego otvara uvodni zaslon koji u nekoliko rečenica objašnjava zašto Slavko bježi i koje tipke upravljaju igrom, a igra počinje tek potvrdom. Četvrti gumb otvara zaslon sa zaslugama, na kojem su navedeni

autor rada, mentor i ustanova, te napomena da su sve slike i zvučni zapisi izrađeni programski za potrebe ovog rada. Oba su zaslona izvedena kao podloge unutar iste scene izbornika, a razred MainMenuController pokazuje jednu po jednu:

```
public void Play()      { Show(introPanel); }  
public void ShowCredits() { Show(creditsPanel); }  
public void BackToMenu() { Show(menuRoot); }
```



Slika 4.7. Prikaz života: dva puna srca i jedno izgubljeno (snimio autor)

Pritiskom tipke Escape igra se pauzira i prikazuje izbornik s mogućnostima nastavka, ponovnog pokretanja razine i povratka u glavni izbornik. Pri pauziranju se protok vremena zaustavlja postavljanjem faktora vremena na nulu. Budući da isti postupak koriste i zasloni poraza, završetka razine i pobjede, pauza provjerava je li koji od njih već otvoren, jer bi inače nastavak vratio protok vremena dok je zaslon još prikazan.

Ista okolnost utjecala je i na pobjedničku animaciju. Kako je pri pobjedi protok vremena zaustavljen, animacija ne može koristiti uobičajeni vremenski pomak između slika, nego mora koristiti pomak neovisan o faktoru vremena. Animacija se sastoji od poskoka naslova i sedamdeset komada konfeta koji padaju uz gravitaciju, lelujanje i vrtnju, a stvaraju se programski umjesto da budu postavljeni u sceni.

4.13. Izgradnja izvršne datoteke

Kako bi se igra mogla pokrenuti izvan razvojnog okruženja, izrađena je izvršna datoteka za operacijski sustav Windows. Izgradnja nije rađena ručno kroz prozor postavki, nego je napisana pomoćna skripta koja dodaje stavku u izbornik razvojnog okruženja. Razlog je ponovljivost: ručna izgradnja ovisi o tome hoće li se pri svakoj izgradnji odabrati ista odredišna mapa i hoće li popis scena biti potpun, dok skripta oba podatka drži u kodu.

```
[MenuItem("Build/Napravi Windows build %#b")]  
public static void BuildWindows()  
{  
    string[] scenes = GetEnabledScenes();  
    ...  
    var options = new BuildPlayerOptions  
    {  
        scenes = scenes,  
        locationPathName = Path.Combine(dir, ExeName),  
        target = BuildTarget.StandaloneWindows64,  
        options = BuildOptions.None  
    };  
};
```

Skripta se nalazi u mapi Assets/Editor. Taj smještaj nije stvar reda, nego uvjet ispravnosti: razredi iz imenskog prostora UnityEditor postoje samo u razvojnem okruženju, a ne i u gotovoj igri, pa bi skripta smještena drugdje bila uključena u izgradnju i prekinula je s pogreškom prevođenja. Mapa Assets/Editor je jedno od mjesta koja okruženje prepoznaje i iz izgradnje isključuje.

Postavke prikaza odabrane su prema načinu na koji se igra pokreće s radne površine. Zadana razlučivost postavljena je na 1280×720 , a način prikaza na prozor promjenjive veličine, pa se igra dvostrukim klikom otvara u prozoru umjesto preko cijelog zaslona. Sadržaj prozora time nije vezan uz razlučivost zaslona, a korisničko sučelje se, zahvaljujući prilagodbi opisanoj u poglavlju 4.12, ispravno skalira i pri punoj razlučivosti. Odabrane postavke izgradnje navedene su u tablici 4.9.

Tablica 4.9. Postavke izgradnje (izradio autor)

Postavka	Vrijednost
Ciljna platforma	StandaloneWindows64
Odredišna mapa	Build/RunSlavkoRun
Izvršna datoteka	RunSlavkoRun.exe
Zadana razlučivost	1280×720
Način prikaza	prozor promjenjive veličine
Veličina izgrađene igre	113 MB

Izgrađena igra sastoji se od izvršne datoteke i popratne mape s podacima, koje se moraju držati zajedno. Mapa s izgrađenom igrom isključena je iz sustava za praćenje izvornog koda, jer je riječ o izvedenom sadržaju koji se u svakom trenutku može ponovno stvoriti iz izvornog koda, a njezino bi uključivanje spremište povećalo za više od sto megabajta pri svakoj izgradnji.

4.14. Izgled gotove igre

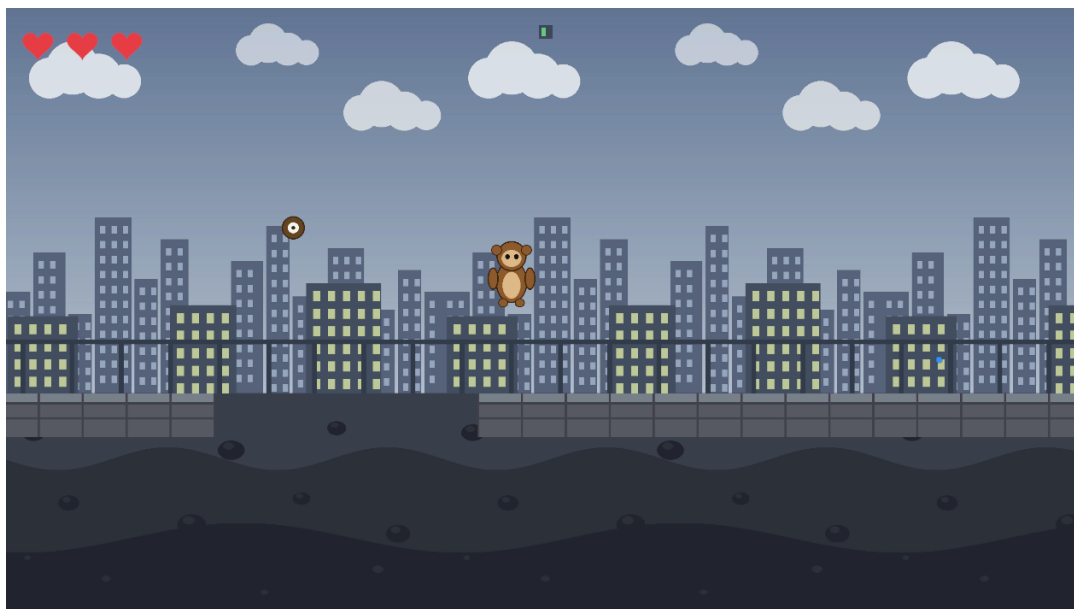
Slike u nastavku prikazuju gotovu igru u radu, snimljene iz izgrađene izvršne datoteke. Glavni izbornik i uvodni zaslone prikazani su na slikama 4.8 i 4.9, druga, treća i četvrta razina na slikama 4.10 do 4.12, ciljna vrata na slici 4.13, a zaslone poraza, pobjede i zasluga na slikama 4.14 do 4.16.



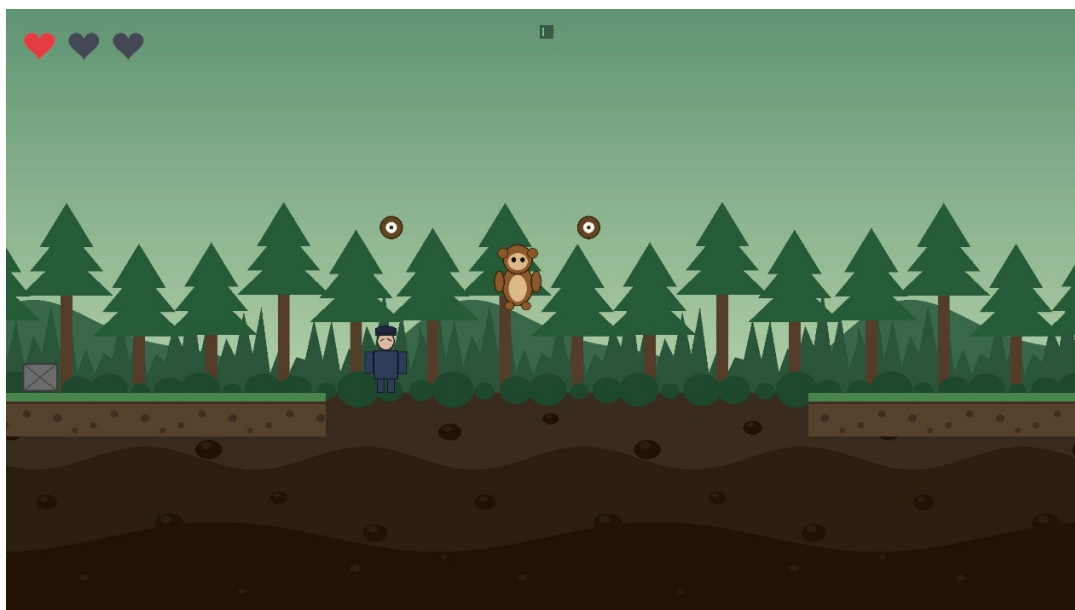
Slika 4.8. Glavni izbornik s četirima mogućnostima i uputama za upravljanje (snimio autor)



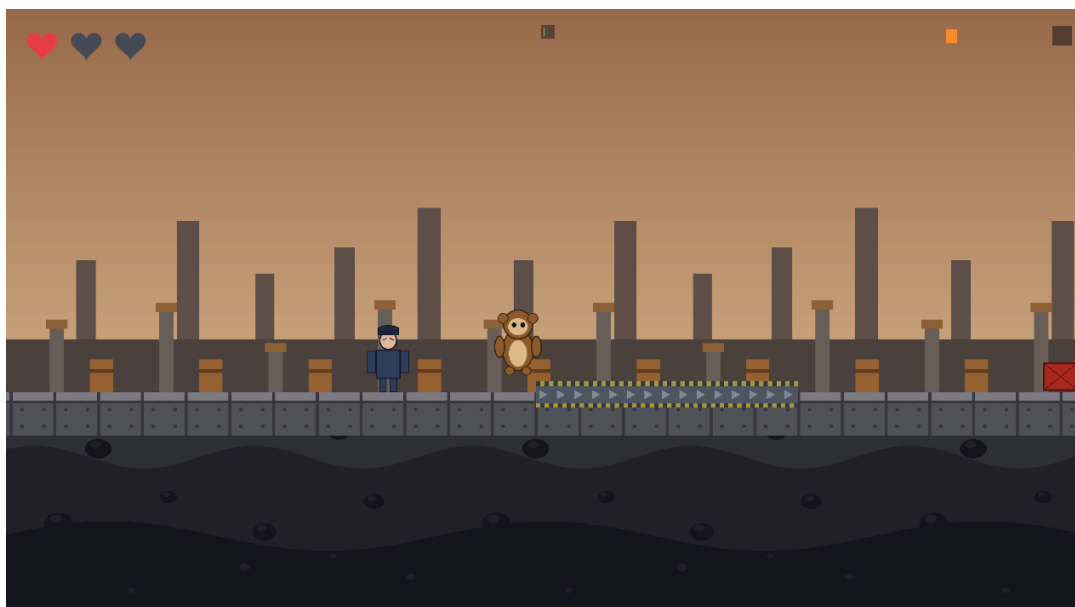
Slika 4.9. Uvodni zaslon prikazan prije prve razine (snimio autor)



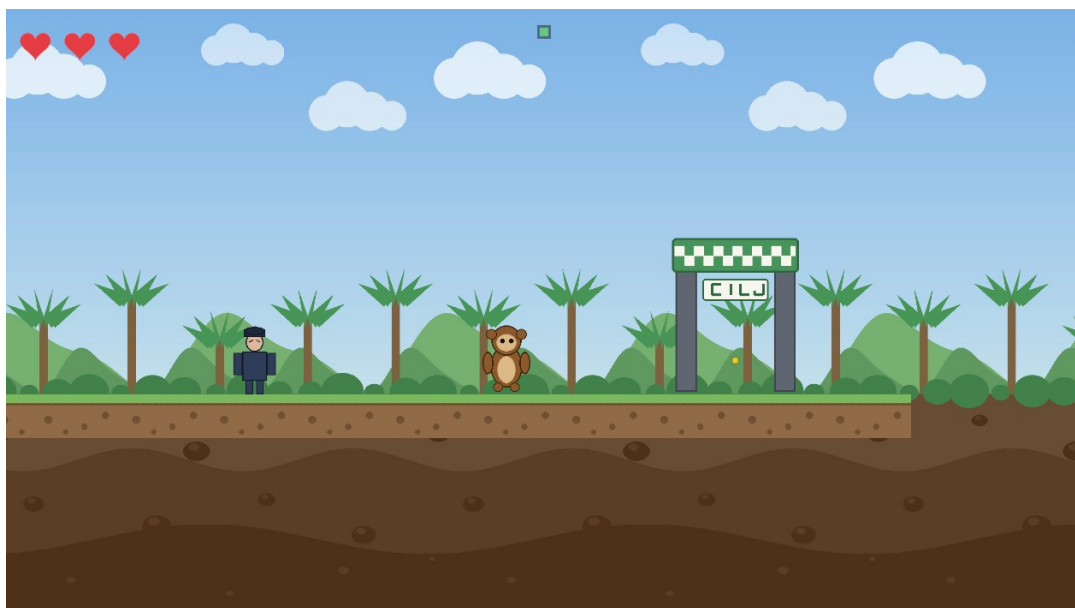
Slika 4.10. Razina 2 (Grad): skok prema točki vješanja iznad jame (snimio autor)



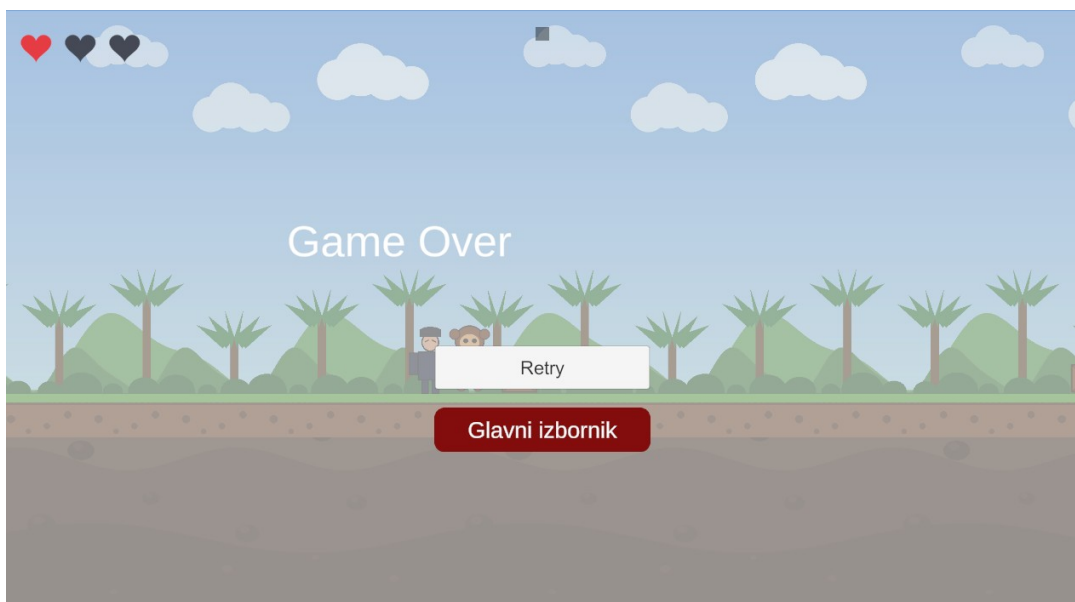
Slika 4.11. Razina 3 (Šuma): lančana jama s dvjema točkama vješanja (snimio autor)



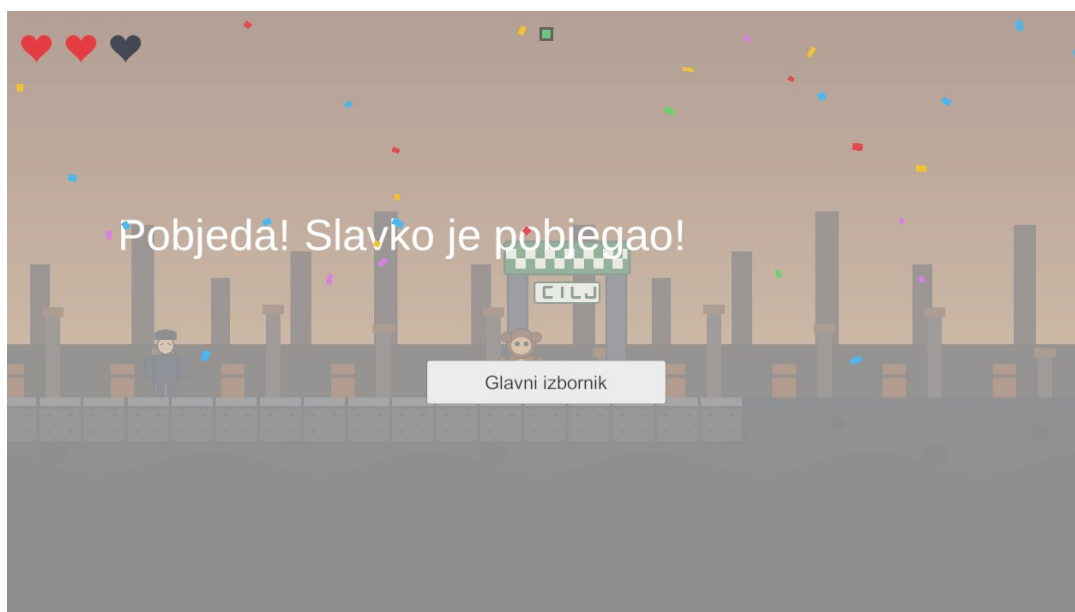
Slika 4.12. Razina 4 (Tvornica): pokretna traka i čuvar neposredno iza (snimio autor)



Slika 4.13. Ciljna vrata kao vidljiv znak da razina završava (snimio autor)



Slika 4.14. Zaslom poraza nakon što je čuvar stigao igrača (snimio autor)



Slika 4.15. Pobjednički zaslon sa sedamdeset komada konfeta (snimio autor)



Slika 4.16. Zaslon sa zaslugama, dostupan iz glavnog izbornika (snimio autor)

5. Testiranje i podešavanje

5.1. Postupak provjere

Provjera ispravnosti provodila se na tri razine. Prvo, cjelovitost datoteka scena provjeravana je programski pri svakoj izmjeni: brojao se broj objekata, provjeravalo se postoje li dvostruke oznake i razrješuju li se sve unutarnje veze. Drugo, prohodnost razina provjeravana je izračunom opisanim u poglavlju 4.7.1. Treće, ponašanje u igri provjeravano je igranjem, jer se osjećaj upravljanja ne može ocijeniti izračunom.

Prve dvije razine provjere izvode se same, pri svakom stvaranju sadržaja, pa se pogreška u zapisu scene otkriva prije nego što se igra uopće pokrene. Treća se izvodi pri svakom sastavljanju razina i

zaustavlja postupak ako neki raspored postane neprohodan. Uz njih je korištena i četvrta, numerička razina: ponašanje čuvara i putanja njihanja provjeravani su simulacijom s istim vremenskim korakom kakav koristi i sama fizika okruženja, čime se ishod tisuću koraka može ocijeniti bez igranja.

Takav raspored ima jasnu granicu. Izračunom se provjerava je li nešto moguće, ali ne i je li ugodno. Podatak da je prozor za hvatanje užeta dug 0,54 sekunde govori da hvatanje jest izvedivo, ali ne i djeluje li ono igraču pravedno; jednako tako, podatak da čuvar u čistoj vožnji ne stiže igrača ne govori stvara li njegova blizina napetost. Ta su pitanja rješavana isključivo igranjem, a izmjene koje su iz njega proizašle opisane su u poglavlju 5.4.

5.2. Značajnije uočene pogreške

U tablici 5.1. dan je pregled značajnijih pogrešaka uočenih tijekom razvoja, njihovih uzroka i načina otklanjanja. Zajedničko im je da se nijedna nije očitovala kao poruka o pogrešci, nego kao neispravno ponašanje čiji uzrok nije bio očit.

Tablica 5.1. Pregled značajnijih pogrešaka (izradio autor)

Očitovanje	Uzrok	Otklanjanje
Uže se ne prikazuje	Iscrtavač linije ne može se dodati na objekt koji već ima iscrtavač sprite-a	Uže se iscrtava na zasebnom objektu djetetu
Lik se lijepi na granu	Zglob HingeJoint2D spaja središte tijela s točkom vješanja	Zamjena zglobom DistanceJoint2D
Točka vješanja nedostižna	Postavljena iznad dosega skoka s ruba jame	Pomaknuta prema naprijed i proširena zona hvatanja
Čuvar prolazi kroz igrača	Oslanjanje na događaj ulaska u okidač, koji se javlja jednom	Tvrdo ograničenje položaja u FixedUpdate
Igrač zaglavljn ispred prepreke	Prepreka ostaje čvrsta nakon sudara	Prepreka se uklanja nakon uspješnog sudara
Sučelje neporavnato pri punoj razlučivosti	Platno postavljeno na stalnu veličinu u pikselima	Prilagodba veličini zaslona uz referencu 1920 × 1080
Gumbi izbornika prestaju raditi	Gumbi upućuju na objekt koji se uništava pri ponovnom učitavanju scene	Gumbi upućuju na skriptu na platnu, koja se ne uništava
Niz pogodaka iz kojeg se igrač ne može izvući	Odbacivanje nakon udarca uvijek prema natrag, dakle prema čuvaru	Smjer odbacivanja određuje izvor pogotka, uz mirovanje čuvara nakon pogotka
Čuvar nikada ne uhvati igrača	Hvatanje se provjeravalo tek pri preklapanju sudarača, a čuvar nije imao trenutak u kojem bi igrač izgubio tlo	Provjera hvatanja po udaljenosti i sudar s preprekom pretvoren u posrtanje koje čuvaru daje priliku
Igrač stoji u jami i ništa se ne događa	Trenje o stijenku jame zadržava lika iznad zone opasnosti	Materijal bez trenja na sudaraču igrača i proširena zona opasnosti
Gumb prekriva tekst na pobjedničkom zaslonu	Pravokutnik teksta zadržao prvotnu veličinu, pa se tekst prelomio u četiri retka	Pravokutnik teksta povećan i pomaknut prema gore

Očitovanje	Uzrok	Otklanjanje
Glazba počinje ispočetka pri ponovnom pokretanju razine	Zahtjev za reprodukcijom zapisa koji već svira	Provjera prije pokretanja reprodukcije

5.3. Dvije pogreške vrijedne posebnog opisa

Dvije pogreške iz tablice 5.1. zaslužuju detaljnije objašnjenje, jer se u obama slučajevima očitovanje činilo posve nepovezano s uzrokom.

5.3.1. Zadržavanje u jami

Igrač koji bi upao u jamu ponekad bi u njoj ostao stajati, bez gubitka života i bez ponovnog pokretanja razine, dok bi čuvar stajao iznad njega. Očitovanje je izgledalo kao da zona opasnosti u dnu jame ne radi.

Provjerom se pokazalo da je uzrok drugdje. Stijenka jame visoka je jednu jedinicu, a lik se prema naprijed kreće postavljanjem brzine, a ne silom. Pri padu u jamu sudarač lika naslanja se na stijenku, a trenje između sudarača zadržava ga na visini od približno $-1,3$ jedinice. Zona opasnosti u to je vrijeme počinjala na visini -5 , dakle $2,89$ jedinica niže, pa je lik ostajao zaglavljen u međuprostoru u kojem ništa nije bilo definirano.

Otklanjanje je stoga imalo dva dijela. Sudaraču igrača pridružen je fizikalni materijal bez trenja, čime lik uz stijenku klizi prema dolje umjesto da se na njoj zadrži, a zona opasnosti proširena je prema gore, tako da sada obuhvaća visine od -6 do -1 i time cijelu dubinu jame. Prvi dio rješava uobičajen slučaj, a drugi osigurava da se, ako lik ipak negdje zapne, ishod dogodi.

5.3.2. Prelomljeni tekst na pobjedničkom zaslonu

Na pobjedničkom zaslonu gumb za povratak u izbornik prekrivao je čestitku. Uzrok je bio propust pri prilagodbi sučelja punoj razlučivosti, opisanoj u poglavlju 4.12: pravokutnici svih elemenata sučelja tada su preračunati, ali pravokutnik pobjedničkog teksta bio je jedini koji je ostao u prvotnoj veličini od 340×50 točaka. Kako je poruka duga približno 1100 točaka, prelomila se u četiri retka koji su izlazili izvan pravokutnika i spuštali se preko gumba. Ispravak je povećanje pravokutnika na 1500×140 točaka i pomak prema gore, čime je između teksta i gumba ostalo 166 točaka razmaka.

Zajedničko je objema pogreškama da se u sučelju i u fizici količine koje se čine nevažnima, poput trenja između dvaju sudarača ili veličine pravokutnika koji je i onako nevidljiv, mogu očitovati kao potpuni izostanak očekivanog ponašanja.

5.4. Podešavanje osjećaja upravljanja

Dio izmjena nije proizašao iz pogrešaka, nego iz osjećaja pri igranju. Najznačajnija takva izmjena bilo je skraćivanje razdoblja gubitka nadzora nakon primljenog udarca. Prvotno je automatsko trčanje bilo isključeno kroz cijelo razdoblje nepovredivosti, koje traje $1,5$ sekundu, pa je igra u tom razdoblju djelovala zamrznuto. Razdvajanjem dviju veličina, tako da nepovredivost i dalje traje $1,5$ sekundu, a gubitak nadzora samo $0,35$ sekundi, zadržana je zaštita nakon udarca bez osjećaja zastoja.

Slično je i s udaljenošću pogleda. Kamera je prvotno prikazivala 10 jedinica po visini, što se pri igranju pokazalo preblizu, jer igrač nije stizao vidjeti nadolazeću prepreku. Povećanjem na 14 jedinica vidno polje prošireno je na približno 25 jedinica po širini. Ta je izmjena zahtijevala i ponovnu izradu pozadina, jer su bile nacrtane točno za prijašnje vidno polje.

6. Zaključak

U radu je izrađena cjelovita dvodimenzionalna platformska igra u razvojnom okruženju Unity, s glavnim izbornikom, četirima razinama, spremanjem napretka, sustavom života, pauzom, zvučnom podlogom i završnom animacijom, izgrađena u samostalnu izvršnu datoteku za Windows. Postavljeni zahtjevi iz poglavlja 3.3. u cijelosti su ispunjeni, a način na koji je ispunjenje svakog od njih utvrđeno naveden je u tablici 3.2.

Najvrjedniji ishod rada nije sama igra, nego pristup njezinu podešavanju. Pokazalo se da se odluke koje se obično donose pokušajima i pogreškama mogu izvesti izračunom. Iz zadanih parametara fizike izračunat je doseg skoka, iz njega su izvedene dopuštene širine jama, a iz putanje skoka položaj točke vješanja. Takav pristup otkrio je i pogrešku koju bi se igranjem teško objasnilo: točka vješanja bila je nedostižna za 0,19 jedinica, što se pri igranju očituje samo kao dojam da mehanika „ne radi”.

Isti se pristup pokazao vrijednim i kod ponašanja protivnika. Dojam da čuvar „nikada ne hvata” mogao se objasniti tek simulacijom, koja je pokazala da razmak ima stabilnu ravnotežnu vrijednost od 2,87 jedinica, dok je za dodir potrebno 0,9. Isti je postupak otkrio i suprotnu krajnost: pokušaj da se čuvar ubrza doveo je do 34 pogotka u jednoj vožnji, jer je odbacivanje nakon udarca guralo igrača ravno u čuvara. Ni jedno ni drugo ponašanje ne bi se moglo pouzdano ocijeniti igranjem, jer se u oba slučaja radi o pojavama koje ovise o zbroju mnogih koraka fizike.

Ta je analiza dovela i do jedine izmjene pravila igre koja nije bila ispravak, nego odluka o dizajnu. Sudar s preprekom više ne oduzima život izravno, nego uzrokuje posrtanje, a život oduzima čuvar ako to izgubljeno tlo uspije iskoristiti. Uloge su time razdvojene: prepreka stvara priliku, a protivnik je iskorištava. Simulacija pokazuje da čista vožnja ne završava hvatanjem, da se jedna pogreška oprašta i da dvije pogreške unutar približno tri sekunde znače gubitak života, čime protivnik prvi put stvarno sudjeluje u ishodu igre.

Zaseban je zaključak donijelo i pitanje razlikovanja razina. Prva je izvedba sve četiri razine sastavljala istim pravilima, pa su se razlikovale samo temom i brzinom, što se pri isprobavanju očitovalo kao dojam da su treća i druga razina jednako teške. Pokazalo se da razliku ne stvara brzina, nego sadržaj: tek kada je svaka razina dobila barem jednu vlastitu mehaniku, a nasljedene mehanike bile pooštrene, napredovanje kroz igru dobilo je smjer. To je ujedno i najizravnija potvrda smjernice iz poglavlja 3.1, prema kojoj težina mora rasti postupno i uvođenjem novoga, a ne pojačavanjem postojećega.

Jednako se pokazala korisnom i automatska provjera prohodnosti. Budući da se izvodi pri svakom sastavljanju razina, izmjena koja bi razinu učinila neprohodnom otkriva se odmah, bez potrebe za igranjem cijele razine. To je posebno važno kod proceduralno sastavljenih razina, gdje izmjena jednog pravila mijenja sadržaj na više desetaka mjesta.

Analiza uočenih pogrešaka pokazuje i jedno šire zapažanje. Većina pogrešaka nije se očitovala kao poruka o pogrešci, nego kao neispravno ponašanje: uže koje se ne prikazuje, gumb koji ne reagira, protivnik koji prolazi kroz igrača. Uzroci su redom bili u pravilima okruženja koja nisu očita, primjerice ograničenje na jedan iscrtavač po objektu ili uništavanje dvostrukog jedinstvenog primjerka pri

ponovnom učitavanju scene. Poznavanje takvih pravila pokazalo se jednako važnim kao i poznavanje samog programskog jezika.

Igra se može proširiti u nekoliko smjerova. Najizravniji je animacija lika, budući da se trenutačno koristi jedna nepomična slika, pa bi ciklus trčanja i zasebne slike za skok i njihanje znatno podigli dojam. Zvučna je podloga izrađena, ali se može proširiti: trenutačno svaka razina ima jednu petlju, dok bi dodavanje kraćeg, napetijeg sloja koji se uključuje kada se čuvar približi dalo povratnu informaciju bez potrebe za pogledom unatrag.

Sadržajno, igra se može proširiti novim vrstama segmenata razine, a s obzirom na to da se razine sastoje od pravila, dodavanje nove vrste segmenta ne zahtijeva izmjenu postojećih razina. Tehnički, sljedeći bi korak bila izgradnja za druge platforme, prije svega za web preglednik putem tehnologije WebGL, jer je skripta za izgradnju već napisana tako da se ciljna platforma mijenja na jednom mjestu.

Literatura

- [1] Compton, K., Mateas, M. Procedural Level Design for Platform Games. Proceedings of the 2nd AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE), 2006., str. 109-111.
- [2] Smith, G., Cha, M., Whitehead, J. A Framework for Analysis of 2D Platformer Levels. Proceedings of the 2008 ACM SIGGRAPH Symposium on Video Games (Sandbox), 2008., str. 75-80.
- [3] Schell, J. The Art of Game Design: A Book of Lenses. 3. izdanje, CRC Press, 2019.
- [4] Unity Technologies. Unity User Manual 6000.0. Dostupno na: <https://docs.unity3d.com/6000.0/Documentation/Manual/>
- [5] Unity Technologies. Unity Scripting API: Rigidbody2D. Dostupno na: <https://docs.unity3d.com/ScriptReference/Rigidbody2D.html>
- [6] Unity Technologies. Unity Scripting API: DistanceJoint2D. Dostupno na: <https://docs.unity3d.com/ScriptReference/DistanceJoint2D.html>
- [7] Unity Technologies. Unity Manual: Canvas Scaler. Dostupno na: <https://docs.unity3d.com/Packages/com.unity.ugui@2.0/manual/script-CanvasScaler.html>
- [8] Unity Technologies. Unity Scripting API: AudioSource. Dostupno na: <https://docs.unity3d.com/ScriptReference/AudioSource.html>
- [9] Microsoft. C# documentation. Dostupno na: <https://learn.microsoft.com/en-us/dotnet/csharp/>
- [10] Chacon, S., Straub, B. Pro Git. 2. izdanje, Apress, 2014. Dostupno na: <https://git-scm.com/book/>
- [11] Saltsman, A. Canabalt. Semi Secret Software, 2009.
- [12] Imangi Studios. Temple Run. Imangi Studios, 2011.
- [13] Halfbrick Studios. Jetpack Joyride. Halfbrick Studios, 2011.
- [14] Ubisoft Montpellier. Rayman Jungle Run. Ubisoft, 2012.

Popis slika

Slika 4.1. Glavni razredi i njihovi odnosi.....	16
Slika 4.2. Tok scena i stanja igre.....	16
Slika 4.3. Anatomija segmenta razine s jamom.....	19
Slika 4.4. Razmak čuvara i igrača kroz vrijeme, za tri načina igranja.....	22
Slika 4.5. Što svaka razina donosi novo.....	26
Slika 4.6. Slojevi pozadine razine Zoo, prikazani jedan do drugog radi provjere bešavnosti spoja....	27
Slika 4.7. Prikaz života: dva puna srca i jedno izgubljeno.....	29
Slika 4.8. Glavni izbornik s četirima mogućnostima i uputama za upravljanje.....	30
Slika 4.9. Uvodni zaslon prikazan prije prve razine.....	31

Slika 4.10. Razina 2 (Grad): skok prema točki vješanja iznad jame.....	31
Slika 4.11. Razina 3 (Šuma): lančana jama s dvjema točkama vješanja.....	32
Slika 4.12. Razina 4 (Tvornica): pokretna traka i čuvar neposredno iza.....	32
Slika 4.13. Ciljna vrata kao vidljiv znak da razina završava.....	33
Slika 4.14. Zaslon poraza nakon što je čuvar stigao igrača.....	33
Slika 4.15. Pobjednički zaslon sa sedamdeset komada konfeta.....	34
Slika 4.16. Zaslon sa zaslugama, dostupan iz glavnog izbornika.....	34

Popis tablica

Tablica 2.1. Značajni paketi u projektu.....	10
Tablica 3.1. Usporedive igre i rješenja preuzeta iz njih.....	12
Tablica 3.2. Zahtjevi i način njihove provjere.....	13
Tablica 4.1. Raspodjela programskog koda po mapama.....	15
Tablica 4.2. Parametri kretanja i izvedene veličine.....	17
Tablica 4.3. Udio uspješnih trenutaka otpuštanja.....	19
Tablica 4.4. Brzine igrača i čuvara po razinama.....	20
Tablica 4.5. Ishod simulacije potjere za različite obrasce pogrešaka.....	22
Tablica 4.6. Rezultat provjere prohodnosti.....	24
Tablica 4.7. Sadržaj razina.....	25
Tablica 4.8. Glazbene petlje po scenama.....	27
Tablica 4.9. Postavke izgradnje.....	30
Tablica 5.1. Pregled značajnijih pogrešaka.....	35